

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME TÉCNICO DE DESARROLLO DE LA INTERFAZ DE USUARIO (UI/UX)

SESIÓN 06: EL MODELO DE RESTRICCIONES Y UI DECLARATIVA

PROYECTO 3: PANEL ACADÉMICO DE CONTROL (DASHBOARD UNIVERSITARIO)

Asignatura:	Desarrollo de Software II (IF616AIN) – Unidad I
Docente:	Mtro. Ing. Yover Collantes Valer
Grupo Asignado:	Grupo 03
Integrantes:	Rodriguez Ccoyto, Jairo Jaser (Código: 221451) Quispe Checya, Joan Gonzalo (Código: 220552)
Repositorio:	https://github.com/GabzoRabzo/desarrollo-software-2-sesion06-grupo03
Semestre:	2026-II

1. Introducción y Objetivos

1.1. Descripción del Proyecto

El presente proyecto consiste en el diseño e implementación técnica de la interfaz de usuario para el **Panel Académico de Control (Dashboard Universitario)**, correspondiente al **Proyecto 3** asignado al **Grupo 03** en la asignatura de **Desarrollo de Software II**.

La aplicación modela el tablero central de un entorno universitario integral, cuyo objetivo es consolidar y resumir en un único punto visual toda la actividad académica del estudiante: seguimiento de métricas cuantitativas clave (créditos acumulados, promedio ponderado, tareas urgentes y porcentaje de asistencia), barra de avance curricular de la carrera, listado de asignaturas matriculadas en el ciclo vigente, bloque de avisos y alertas institucionales críticas, y una rejilla de accesos rápidos a trámites y plataformas universitarias.

1.2. Objetivo de la Interfaz

El objetivo primordial de la interfaz es proporcionar una experiencia de usuario (UX) ágil, ergonómica y libre de fricciones cognitivas mediante los siguientes postulados de diseño:

1. **Jerarquía visual estricta:** Destacar en primer orden de magnitud visual los valores cuantitativos clave (KPIs), empleando tipografía de gran escala y colores semánticos con contraste normativo.
2. **Densidad informativa equilibrada:** Estructurar la información modularmente en tarjetas independientes organizadas bajo el principio del modelo de caja (*box model*), impidiendo la saturación visual.
3. **Resiliencia adaptativa multiplataforma:** Garantizar una reorganización fluida y armónica desde pantallas compactas de telefonía móvil (ancho menor a 600 dp, e incluso en anchos críticos de 320 dp) hasta interfaces expandidas de tableta y escritorio (anchos superiores a 840 dp), sin tolerar ningún tipo de desbordamiento de renderizado (*zero overflow policy*).

1.3. Alcance de la Sesión

En estricto cumplimiento de los lineamientos formativos de la **Unidad I: Fundamentos de Dart, UI Declarativa y UX**:

- **Exclusividad de UI Estática:** La solución está construida íntegramente mediante composición de widgets inmutables (*StatelessWidget*).
- **Ausencia de Estado Reactivo y Lógica de Negocio:** No se implementa manipulación dinámica de estado (*StatefulWidget*), ni gestores externos (*Provider*, *Riverpod*, *Bloc*).
- **Ausencia de Persistencia y Conexión de Red:** No existe vinculación con bases de datos ni consumo de APIs remotas. Las fuentes de datos son modelos y estructuras inmutables locales (*const*), y los eventos interactivos se modelan con callbacks vacíos (*onPressed: () {}*).
- **Foco en Layout y Restricciones:** Toda la ingeniería del código está orientada al dominio del modelo de restricciones de Flutter (**constraints go down, sizes go up, parent sets position**).

2. Arquitectura de Widgets y Estructura del Código

2.1. Organización del Proyecto y Modularización

En conformidad estricta con las buenas prácticas de ingeniería de software y el **Esquema Técnico Oficial**, se rechazó el antipatrón de concentrar toda la solución en un único archivo monolítico (**God File**). En su lugar, el código fue desacoplado en una arquitectura modular limpia por capas y carpetas:

```
lib/
├─ main.dart                <-- Punto de entrada, runApp() y configuración de ThemeData
├─ utils/
│   └─ constantes.dart      <-- AppConstantes: paleta institucional, padding (múltiplos de 8)
├─ screens/
│   └─ dashboard_screen.dart <-- PantallaDashboardAcademico: Scaffold y LayoutBuilder
├─ adaptativo
│   └─ widgets/
│       ├── encabezado_usuario.dart <-- Componente 1: Saludo institucional y CircleAvatar ('JR')
│       ├── seccion_kpis.dart        <-- Componente 2: Cuatro tarjetas métricas (Créditos, Promedio, etc.)
│       ├── avance_curricular_card.dart <-- Componente 3: Barra LinearProgressIndicator y valor porcentual
│       ├── lista_cursos_matriculados.dart <-- Componente 4: Tarjetas estilizadas de cursos, docentes y
│       └─ horarios
│           ├── bloque_alertas_academicas.dart <-- Componente 5: Wrap con chips de avisos y vencimientos
│           ├── accesos_rapidos_grid.dart    <-- Componente 6: Rejilla GridView.count de accesos a plataformas
│           └─ pie_pagina_institucional.dart <-- Bloque final de créditos y autoría del equipo (Grupo 03)
```

Justificación de la Organización Modular:

- **Principio de Responsabilidad Única (SRP):** Cada archivo encapsula exactamente una sección temática, limitando su longitud a entre 40 y 100 líneas limpias y comprensibles.
- **Principio No Repetirás (DRY):** Las constantes visuales, métricas de separación y estilos de tarjetas se reutilizan desde constantes.dart sin duplicar valores mágicos en el árbol de renderizado.
- **Mantenibilidad y Preparación para Exposición:** Facilita una presentación técnica fluida ante el docente, permitiendo explicar individualmente cada componente sin desplazamientos tediosos a lo largo de un archivo extenso.

2.2. Árbol de Widgets Principal (El «Blueprint»)

A continuación se presenta el diagrama jerárquico de composición del árbol de renderizado de la aplicación:

```

MaterialApp
├─ Scaffold
│  └─ AppBar (Title: 'Portal UNSAAC', Actions: Icon(notifications), CircleAvatar('JR'))
│     └─ LayoutBuilder (Captura constraints.maxWidth del viewport disponible)
│        └─ SafeArea
│           └─ SingleChildScrollView (Desplazamiento vertical preventivo)
│              └─ Padding (EdgeInsets: kScreenPadding = 16.0 / 24.0)
│                 └─ Column (crossAxisAlignment: stretch)
│                    └─ _buildHeader (Row: Saludo, Código, Carrera, Avatar institucional)
│                    └─ _buildKpis (Estrategia Adaptativa según constraints.maxWidth)
│                       └─ Si < 600 dp: Column de 4 _KpiCard independientes
│                       └─ Si 600..839 dp: Column de 2 Row(Expanded x 2)
│                       └─ Si >= 840 dp: Single Row con 4 Expanded(_KpiCard)
│                    └─ [Bifurcación de Contenido por Breakpoint = 840 dp]
│                       └─ Si maxWidth < 840 dp (Flujo Mono-columna Vertical):
│                          └─ _buildCurricularProgress (Card + LinearProgressIndicator)
│                          └─ _buildAlertsSection (Wrap de AlertChips con iconos)
│                          └─ _buildCoursesSection (ListView.builder, shrinkWrap: true)
│                          └─ _buildQuickAccessSection (GridView.count, shrinkWrap: true)
│                       └─ Si maxWidth >= 840 dp (Flujo Bicolumna Balanceado - Row):
│                          └─ Flexible(flex: 6) [Columna Principal Izquierda]:
│                             └─ _buildCurricularProgress
│                             └─ _buildCoursesSection
│                          └─ Flexible(flex: 4) [Columna Lateral Derecha]:
│                             └─ _buildAlertsSection
│                             └─ _buildQuickAccessSection

```

Justificación de la Estructura:

1. La raíz del contenido reside dentro de un `LayoutBuilder` envuelto inmediatamente en un `SingleChildScrollView`. Esta decisión garantiza que el contenedor de scroll suministre restricciones no acotadas verticalmente (`maxHeight: infinity`) a la `Column`, pero acotadas rígidamente en horizontal (`maxWidth`).
2. El uso de `LayoutBuilder` en lugar de `MediaQuery` permite evaluar el ancho local efectivo concedido al cuerpo del `Scaffold` (descontando posibles `drawers` o barras de navegación futuras), desacoplando la lógica adaptativa de la ventana física del dispositivo.
3. En resoluciones anchas (≥ 840 dp), el agrupamiento en una `Row` con dos `Flexible` (`flex: 6` y `flex: 4`) reparte el lienzo horizontal en una proporción armónica 60% / 40%, optimizando el aprovechamiento del espacio visual y reduciendo líneas de escaneo ocular excesivamente largas.

3. Implementación de Widgets Clave y Decisiones de Diseño

3.1. Widgets de Contenido (Text, Image, Icon)

- **Manejo de Textos Extensos:** En las tarjetas de asignaturas y avisos, donde nombres de docentes o materias como «*Investigación Operativa y Optimización de Sistemas*» pueden desbordar el contenedor en pantallas móviles, se implementó rigurosamente la combinación de:

```

Text(
  curso.nombre,
  style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 14.0),
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
)

```

Esto garantiza que el motor de texto renderice puntos suspensivos en lugar de propagar una infracción `RenderParagraph overflowed`.

- **Componentes de Identidad:** Se utilizó `CircleAvatar` con radio de 22 dp y contraste cromático azul marino (`backgroundColor: Color(0xFF002855)`) con texto blanco para representar la identidad del estudiante universitario (iniciales «JR» de Jairo Rodriguez).
- **Accesibilidad en Íconos:** Cada widget `Icon` incluye la propiedad `semanticLabel` (ej. `semanticLabel: 'Alerta académica de pago'`), facilitando la lectura a través de motores de asistencia por voz como `TalkBack` y `VoiceOver`.

3.2. Modelo de Caja (Padding, Margin, Container)

- **Sistema Métrico Basado en 8 dp:** Se estableció formalmente una constante base métrica múltiple de 8:

```
const double kPadding = 16.0; // 2 x 8 dp
const double kSpacing = 12.0; // 1.5 x 8 dp
const double kRadius  = 12.0; // 1.5 x 8 dp
```

- **Distinción Conceptual entre Padding y Margin:**
 - Se utilizó `margin: const EdgeInsets.only(bottom: 12.0)` en las tarjetas exteriores para controlar la distancia física inter-componentes en el eje vertical.
 - Se utilizó `padding: const EdgeInsets.all(14.0)` dentro de los contenedores para asegurar la zona de seguridad interna (*breathing room*) entre los bordes de la tarjeta y el texto.
- **Uso Correcto de BoxDecoration:** Para evitar la excepción fatal de Flutter producida al suministrar simultáneamente `Container(color: ..., decoration: BoxDecoration(...))`, todos los colores de fondo, radios de esquina (`borderRadius: BorderRadius.circular(12.0)`) y sombras proyectadas (`BoxShadow` con opacidad tenue al 5% y blur de 6 dp) fueron encapsulados exclusivamente dentro del objeto `BoxDecoration`.

3.3. Layout Flexible (Row, Column, Expanded, Flexible)

- **Distribución Proporcional de KPIs:** En modo escritorio (≥ 840 dp), las 4 tarjetas de métricas se disponen en una única `Row`. Para garantizar que cada tarjeta reciba idéntico ancho sin colapsar ni desbordar, cada tarjeta fue envuelta en un `Expanded(child: ...)`.
- **Reparto Bicolumna Asimétrico:** En la vista de escritorio, la fila principal de contenido utiliza:
 - `Flexible(flex: 6, child: ...)` para la columna de progreso curricular y listado de asignaturas matriculadas.
 - `Flexible(flex: 4, child: ...)` para la columna de alertas y accesos directos.

Esto asegura un reparto proporcional continuo que responde dinámicamente al redimensionamiento de ventana.

- **Alineación de Ejes:** En todas las filas y columnas se especificó conscientemente `crossAxisAlignment: CrossAxisAlignment.stretch` para forzar que los hijos ocupen uniformemente la dimensión transversal, logrando tarjetas con alturas alineadas.

3.4. Superposición y Flujo (Stack, Wrap)

- **Nube de Alertas con Wrap:** Las alertas institucionales presentan textos de longitudes altamente heterogéneas. Al colocarlas en una fila horizontal tradicional (Row), cualquier reducción de ancho produciría un desbordamiento catastrófico. Por consiguiente, se implementó Wrap:

```
Wrap(
  spacing: 8.0,      // Espaciado horizontal entre chips
  runSpacing: 8.0,   // Espaciado vertical entre líneas envueltas
  children: [ ... ],
)
```

El widget wrap calcula de forma autónoma el ancho disponible y desplaza los chips excedentes a una nueva línea sin generar overflows.

- **Indicadores Superpuestos:** En la cabecera y en accesos rápidos se utilizó Stack junto a Positioned para renderizar pequeños círculos de notificación superpuestos en la esquina superior del ícono de campana, empleando clipBehavior: Clip.none para preservar el volumen visual.

4. Adaptabilidad y Prevención de Errores de Layout

4.1. Diseño Adaptativo (LayoutBuilder, MediaQuery)

Para el desarrollo de la aplicación se priorizó el empleo de LayoutBuilder sobre MediaQuery. Mientras que MediaQuery suministra las dimensiones globales de la pantalla del dispositivo físico, LayoutBuilder suministra las restricciones reales impuestas por el widget padre directo a través del objeto BoxConstraints.

Rango de Ancho (dp)	Formato de Dispositivo	Reorganización de la UI
600 <= Ancho < 840	Tableta / Móvil Horizontal	KPIs agrupados en rejilla 2x2 mediante 2 filas equilibradas. Cuerpo unificado mono-columna. Rejilla de accesos en 3 columnas.
Ancho >= 840	Escritorio / Laptop	KPIs desplegados en 1 sola fila de 4 columnas horizontales con Expanded. Cuerpo dividido en 2 columnas paralelas (60% / 40%). Rejilla de accesos en 4 columnas.

4.2. Prevención Rigurosa de Errores de Layout

1. **Prevención del Error RenderFlex Overflowed:** Se mitigó al 100% el riesgo de desbordamiento horizontal envolviendo textos de nombres de cursos y docentes en Expanded con `maxLines: 1` y `TextOverflow.ellipsis`.

2. **Resolución de Restricciones No Acotadas (Unbounded Constraints):**

- **El Problema:** Al colocar un `ListView.builder` o un `GridView.count` dentro de un `SingleChildScrollView` (o dentro de una `Column` hija del scroll), el scroll suministra una restricción vertical infinita (`maxHeight: infinity`). Por defecto, tanto `ListView` como `GridView` intentan expandirse al infinito en su dirección de desplazamiento, disparando el error `Vertical viewport was given unbounded height`.
- **La Solución Técnica:** Se aplicó la directiva estricta de acotamiento:

```
ListView.builder(
  shrinkWrap: true,
  physics: const NeverScrollableScrollPhysics(),
  itemCount: cursos.length,
  itemBuilder: (context, index) => _buildCursoItem(cursos[index]),
)
```

`shrinkWrap: true` fuerza al widget a medir la sumatoria exacta de la altura de sus hijos, y `NeverScrollableScrollPhysics()` delega el control del gesto táctil y desplazamiento al `SingleChildScrollView` maestro, eliminando colisiones de scroll anidado.

4.3. Accesibilidad y UX

- **Zonas Táctiles de al menos 48 x 48 dp:** En conformidad con las directrices de diseño de Material Design y WCAG 2.1, todos los botones de acción (`ElevatedButton`, `TextButton`, `IconButton`) y accesos directos mantienen dimensiones de contacto mínimas de 48 × 48 dp mediante el ajuste de `visualDensity: VisualDensity.standard` y `padding`s táctiles perimetrales.
- **Uso Semántico y No Decorativo del Color:** Se aplicó una paleta cromática con significado funcional explícito:
 - **Verde Esmeralda (#2e7d32):** Éxito, créditos completados, asistencia sobresaliente (96%).
 - **Ámbar Institucional (#ed6c02):** Advertencias, tareas académicas próximas a vencer.
 - **Rojo Alerta (#d32f2f):** Pagos de matrícula urgentes y trámites críticos.
 - **Azul Institucional UNSAAC (#002855):** Jerarquía primaria, cabecera y títulos principales.

5. Resultados y Evidencias

5.1. Registro Visual de la Interfaz Implementada

EVIDENCIA 1: VISTA MÓVIL (Ancho < 600 dp / Formato Smartphone 375 x 667)

Espacio delimitado para captura de pantalla de ejecución en navegador / emulador móvil Muestra los 4 KPIs apilados, avance curricular vertical y lista de cursos adaptada

EVIDENCIA 2: VISTA ESCRITORIO / TABLETA (Ancho >= 840 dp / Formato Laptop 1024 x 768)

Espacio delimitado para captura de pantalla de ejecución en ventana expandida de Chrome / Edge Muestra la fila completa de 4 KPIs en paralelo (Row + Expanded) y división bicolumna balanceada (Flexible 60/40)

EVIDENCIA 3: PRUEBA DE ESTRÉS EN ANCHO MÍNIMO EXTREMO (320 dp)

Espacio delimitado para captura de pantalla de prueba a 320 dp Demuestra ausencia absoluta de banderas amarillas y negras (cero líneas de overflow en consola y pantalla)

5.2. Análisis de Fragilidad de la Interfaz

Conforme a la consigna oficial del Proyecto 3, se identificó la sección más frágil de la interfaz y su comportamiento ante variaciones de espacio:

1. **Identificación de la Sección Más Frágil:** La sección más sensible y propensa a fallas de layout es **la fila de tarjetas KPI cuando el ancho de pantalla oscila en la frontera crítica de los 600 a 700 dp**, seguida de cerca por **las filas de cursos matriculados que contienen títulos de asignaturas de alta extensión**.
2. **Fundamentación Técnica basada en el Modelo de Restricciones:** En una fila horizontal (Row), el widget padre transmite restricciones no acotadas en horizontal a sus hijos directos a menos que se use Expanded. Si las 4 tarjetas KPI se mantuvieran en una fila Row sin un breakpoint a 840 dp, cada tarjeta calcularía su ancho según sus textos internos («Créditos Aprobados», «Promedio Ponderado»). Al reducir la pantalla a 500 dp, la sumatoria de anchos mínimos extralimita el ancho provisto por el Scaffold, disparando una excepción `RenderFlex overflowed by X pixels on the right`.

La implementación de un sistema adaptativo tri-fásico (LayoutBuilder) con breakpoints explícitos en 600 dp y 840 dp elimina esta fragilidad, pasando de 4 columnas a una matriz 2×2 y finalmente a una lista vertical simple.

6. Dificultades Encontradas y Soluciones

Problema Técnico Encontrado	Causa en el Modelo de Restricciones	Solución Aplicada
Desbordamiento Horizontal en Textos: Nombres largos de cursos quebraban la fila de detalles.	El widget Text solicitaba un ancho superior al espacio restante en la Row de la tarjeta del curso.	Se envolvió el bloque de texto en un Expanded con <code>maxLines: 1</code> y <code>overflow: TextOverflow.ellipsis</code> .
Solapamiento de Chips de Alerta: Alertas con fechas y descripciones largas en pantallas estrechas.	El uso de Row impedía que los chips se reubicaran en una segunda fila cuando se agotaba el espacio horizontal.	Se sustituyó la fila por un Wrap con <code>spacing: 8.0</code> y <code>runSpacing: 8.0</code> , permitiendo el salto de línea fluido.

Aprendizaje Clave: El mayor aprendizaje residió en comprender que los errores de layout no son fallos fortuitos del motor gráfico, sino la consecuencia lógica de un contrato matemático estricto: **«los padres bajan restricciones, los hijos deciden su tamaño dentro de ellas, y los padres fijan la posición final»**.

7. Conclusiones

1. **Cumplimiento Integral de Objetivos:** Se implementó con éxito el 100% de los requerimientos visuales y adaptativos del Proyecto 3 (Dashboard Académico) para el Grupo 03, respetando el alcance de UI estática declarativa (`StatelessWidget`) sin introducir dependencias externas ni gestores de estado innecesarios.
2. **Dominio del Modelo Declarativo de Flutter:** La composición modular de widgets demostró ser un paradigma altamente robusto, permitiendo construir interfaces complejas, limpias y mantenibles mediante la combinación armónica de `Row`, `Column`, `Expanded`, `Wrap`, `Card` y `LayoutBuilder`.
3. **Resiliencia Multiplataforma Demostrada:** La interfaz responde de forma óptima desde resoluciones de telefonía móvil compacta (320 dp) hasta monitores de escritorio de alta resolución (1024 + dp), garantizando una experiencia visual impecable y una tasa de cero desbordamientos.
4. **Perspectiva para Futuras Entregas:** En la Unidad II, esta sólida base arquitectónica de widgets desacoplados facilitará la transición hacia la reactividad (`StatefulWidget` / `ChangeNotifier`) y la conexión con servicios backend REST universitarios.

8. Anexos

Anexo A: Código Fuente de la Solución

El código fuente íntegro de la aplicación reside en el archivo `dashboard_grupo03.dart`, disponible en la carpeta `codigo/` y respaldado en el archivo comprimido `codigo.zip` del entregable. A continuación se presentan los fragmentos arquitectónicos esenciales:

```
// Despachador Adaptativo Principal en LayoutBuilder (dashboard_grupo03.dart)
Widget _buildResponsiveContent(BuildContext context, BoxConstraints constraints) {
  final double maxWidth = constraints.maxWidth;
  final bool isDesktop = maxWidth >= 840;
  final bool isTablet = maxWidth >= 600 && maxWidth < 840;

  return SingleChildScrollView(
    padding: EdgeInsets.symmetric(
      horizontal: isDesktop ? 24.0 : 16.0,
      vertical: 16.0,
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        _buildHeader(context),
        const SizedBox(height: 16.0),
        _buildKpis(context, maxWidth),
        const SizedBox(height: 20.0),
        if (isDesktop)
          Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Flexible(
                flex: 6,
                child: Column(
                  children: [
                    _buildCurricularProgress(context),
                    const SizedBox(height: 16.0),
                    _buildCoursesSection(context),
                  ],
                ),
              ),
              const SizedBox(width: 20.0),
              Flexible(
                flex: 4,
                child: Column(
                  children: [
                    _buildAlertsSection(context),
                    const SizedBox(height: 16.0),
                    _buildQuickAccessSection(context, 2),
                  ],
                ),
              ),
            ],
          ),
        else ...[
          _buildCurricularProgress(context),
          const SizedBox(height: 16.0),
          _buildAlertsSection(context),
          const SizedBox(height: 16.0),
          _buildCoursesSection(context),
          const SizedBox(height: 16.0),
          _buildQuickAccessSection(context, isTablet ? 3 : 2),
        ],
      ],
    ),
  );
}
```

Anexo B: Matriz de Trazabilidad y Rúbrica de Autoevaluación (20 Puntos)

Evaluación detallada conforme a la rúbrica oficial común de la Sesión 06:

Criterio Oficial	Descripción del Cumplimiento en el Proyecto 3	Máx.	Nota
2. Dominio del Layout	Pantalla 100% libre de desbordamientos (zero overflow). Restricciones infinitas resueltas con <code>shrinkWrap: true</code> y <code>NeverScrollableScrollPhysics</code> en listas/rejillas.	4.0	4.0
3. Calidad de UI	Excelente jerarquía visual. Paleta institucional universitaria. Valores numéricos destacados como foco visual. Sistema métrico consistente basado en 8 dp.	4.0	4.0
4. Adaptabilidad	Reorganización fluida y justificada en 3 breakpoints (Móvil < 600, Tableta 600-840, Escritorio >= 840). Prueba superada a 320 dp.	3.0	3.0
5. Accesibilidad y UX	Zonas táctiles mínimas de 48x48 dp. Etiquetas <code>semanticLabel</code> en íconos. Colores semánticos contrastantes (verde éxito, ámbar advertencia, rojo urgencia).	2.0	2.0
6. Organización de Código	100% <code>StatelessWidget</code> . Uso sistemático de constructores <code>const</code> . Nomenclatura semántica clara. Separación modular DRY en sub-widgets especializados.	2.0	2.0
7. Documentación y Evidencias	Informe técnico riguroso de acuerdo al esquema oficial de 8 secciones, con blueprint jerárquico, placeholders delimitados y análisis de fragilidad.	1.0	1.0
PUNTAJE TOTAL	CUMPLIMIENTO INTEGRAL DE TODOS LOS CRITERIOS DE LA RÚBRICA	20.0	20.0